

Week 2 Project

Production SQL Serving Stack

AWQ Quantize · vLLM Deploy · FastAPI · Prometheus · SLO Benchmark

WEEK 2 · DAY 14

Sunday, July 19 2026

Day 14 of 84

Step	What	Details
Step 1	AWQ Quantize	FP16 → INT4: 16 GB → 4 GB; W4A16; calibration dataset
Step 2	vLLM Deploy	All efficiency flags: prefix cache, chunked prefill, FP8 KV
Step 3	FastAPI API	/sql, /health, /evaluate, /metrics endpoints
Step 4	Prometheus	Counter, Histogram, Gauge for latency, errors, tokens
Step 5	SLO Benchmark	TTFT p95 < 500ms, e2e p99 < 2s, error < 1%
Step 6	Monitoring Stack	Docker Compose: Prometheus + Grafana dashboards
Step 7	Readiness Check	Automated go/no-go before deployment
Step 8	FAANG Patterns	Meta ramp, Google canary, Amazon variants, Netflix shadow

ARCHITECTURE: Fine-tuned SQL model (Day 9) → AWQ INT4 (Day 8) → vLLM server (Day 11+13 flags) → FastAPI wrapper (/sql, /health, /metrics, /evaluate) → Prometheus + Grafana → SLO gate → production canary. This is Capstone Piece 5 of 8 — the serving layer for the Text-to-SQL system.

Section 1 — System Architecture

CLIENT REQUEST FLOW:

```

User question
|
FastAPI /sql endpoint (port 9000)
|
+-- schema lookup (in-memory registry)
|
+-- build prompt: system + schema + question
|
+-- POST http://localhost:8000/v1/chat/completions (vLLM)
|   |
|   +-- prefix cache check (system + schema)
|   |   | HIT: skip prefill, use cached KV -> TTFT < 50ms
|   |   | MISS: full prefill via chunked prefill -> TTFT ~300ms
|   |
|   +-- decode: generate SQL tokens (FP8 KV cache, AWQ weights)
|   |
|   +-- return completion JSON
|
+-- sqlparse syntax check
+-- optional: execute on in-memory SQLite DB
+-- record Prometheus metrics
+-- return SQLResponse {sql, tokens, latency_ms, ttft_ms, valid_syntax}

```

ENDPOINTS:

```

POST /sql          Generate SQL from natural language
GET  /health       Liveness check (verifies vLLM is up)
GET  /metrics       Prometheus scrape endpoint
GET  /evaluate      Run mini regression suite (execution accuracy)

```

Section 2 — Step 1: AWQ Quantization

AWQ (Activation-aware Weight Quantization) quantizes model weights to INT4 while keeping activations at FP16 (W4A16). A calibration dataset guides which weight channels are most salient — those get protected during quantization. Result: 4× memory reduction at < 1% quality loss.

```

from awq import AutoAWQForCausalLM
from transformers import AutoTokenizer

BASE_MODEL = "./sql-llama-8b-merged" # Day 9 LoRA-merged model
OUTPUT_DIR = "./sql-llama-8b-awq"

quant_config = {
    "zero_point": True, # asymmetric quantization
    "q_group_size": 128, # 128 weights share one scale factor
    "w_bit": 4, # 4-bit weights → W4A16
    "version": "GEMM", # GEMM kernel (fastest on A100/A10G)
}

model = AutoAWQForCausalLM.from_pretrained(BASE_MODEL, low_cpu_mem_usage=True)
tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL)

# Calibration data: 160 SQL examples from training set

```

```
# AWQ uses these to identify outlier activation channels
calib_data = [
    f"<|system|>\nSQL expert. Schema:{SCHEMA}\n<|user|>\n{q}\n<|assistant|>\n{a}"
    for q, a in sql_train_pairs[:160]
]

model.quantize(tokenizer, quant_config=quant_config, calib_data=calib_data)
model.save_quantized(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)

# Size verification:
# FP16 LLaMA 3 8B: ~16 GB
# AWQ INT4: ~4 GB (4x reduction)
# Quality: FP16 = 91% execution acc → AWQ = 90.1% (<1% drop)
```

Format	Size (8B)	Quality	Method	When to Use
FP16 / BF16	16 GB	100% baseline	No quantization	Training only
AWQ INT4	4 GB	99% of FP16	W4A16, absmax groups	Production serving (best)
GPTQ INT4	4 GB	98-99%	W4A16, Hessian-based	Alternative to AWQ
NF4 (QLoRA)	4 GB	99%	Normal float 4-bit	Fine-tuning only
GGUF Q4_K_M	4.5 GB	98-99%	K-quant mixed	CPU / Apple Silicon

Section 3 — Step 2: vLLM Production Deployment

```
#!/bin/bash
# serve.sh — start vLLM with all Week 2 efficiency flags (Day 13)

python -m vllm.entrypoints.openai.api_server \
    --model ./sql-llama-8b-awq \
    --quantization awq \
    --dtype float16 \
    --tensor-parallel-size 1 \           # single A100 for 8B AWQ
    --gpu-memory-utilization 0.90 \
    --max-model-len 4096 \
    --max-num-seqs 256 \               # max concurrent sequences
    --enable-prefix-caching \         # Day 13: cache system prompt + schema
    --enable-chunked-prefill \        # Day 13: interleave prefill + decode
    --max-num-batched-tokens 2048 \
    --kv-cache-dtype fp8 \            # Day 13: 2x KV memory
    --port 8000 --host 0.0.0.0 \
    --served-model-name sql-llama-8b \
    --disable-log-requests            # reduce log volume in production

# Efficiency profile (single A100 80GB):
# Weights: 4.0 GB (AWQ INT4)
# KV cache budget: 68.0 GB (90% of 80GB - 4GB weights)
# KV per request: 131 MB (4096 ctx, FP8)
# Max concurrent: ~518 requests
# Throughput (bs=32): 280-320 tokens/sec
# TTFT (cache hit): < 50ms
# TTFT (cache miss): ~300ms
```

Section 4 — Step 3: FastAPI Production Endpoint

The FastAPI layer wraps vLLM with business logic: schema lookup, syntax validation, optional SQL execution, Prometheus metrics, and health checks.

```
from fastapi import FastAPI, HTTPException
from fastapi.responses import PlainTextResponse
from pydantic import BaseModel
from prometheus_client import Counter, Histogram, Gauge, generate_latest
import httpx, time, sqlparse, sqlite3

app = FastAPI(title="SQL Generation API", version="1.0.0")

# Prometheus metrics
REQUEST_COUNT = Counter("sql_requests_total", "Total requests", ["status", "schema_name"])
LATENCY_HIST = Histogram("sql_latency_seconds", "E2E latency",
    buckets=[.05, .1, .2, .5, 1, 2, 5])
TTFT_HIST = Histogram("sql_ttft_seconds", "Time to first token",
    buckets=[.02, .05, .1, .2, .5, 1])
TOKENS_HIST = Histogram("sql_output_tokens", "Output tokens",
    buckets=[10, 25, 50, 100, 200, 400])
SYNTAX_ERRORS = Counter("sql_syntax_errors_total", "Syntax errors")
ACTIVE_REQUESTS = Gauge("sql_active_requests", "In-flight requests")

class SQLRequest(BaseModel):
```

```

    question:    str
    schema_name: str = "ecommerce"
    temperature: float = 0.05
    execute:     bool = False

class SQLResponse(BaseModel):
    sql:         str
    tokens:      int
    latency_ms:  float
    ttft_ms:     float
    valid_syntax: bool
    rows:        list | None = None

@app.post("/sql", response_model=SQLResponse)
async def generate(req: SQLRequest):
    if req.schema_name not in SCHEMAS:
        raise HTTPException(404, f"Unknown schema: {req.schema_name}")
    ACTIVE_REQUESTS.inc()
    t0 = time.perf_counter()
    try:
        result = await call_vllm(req.question, SCHEMAS[req.schema_name], req.temperature)
    except Exception as e:
        REQUEST_COUNT.labels(status="error", schema_name=req.schema_name).inc()
        raise HTTPException(502, str(e))
    finally:
        ACTIVE_REQUESTS.dec()

    valid = is_valid_sql(result["sql"])
    if not valid: SYNTAX_ERRORS.inc()
    rows = execute_sql(result["sql"], req.schema_name) if req.execute else None

    wall_ms = (time.perf_counter() - t0) * 1000
    REQUEST_COUNT.labels(status="ok", schema_name=req.schema_name).inc()
    LATENCY_HIST.observe(wall_ms / 1000)
    TTFT_HIST.observe(result["ttft_ms"] / 1000)
    TOKENS_HIST.observe(result["tokens"])
    return SQLResponse(**result, latency_ms=wall_ms, valid_syntax=valid, rows=rows)

@app.get("/health")
async def health():
    try:
        async with httpx.AsyncClient(base_url="http://localhost:8000", timeout=5) as c:
            r = await c.get("/health")
            vllm_ok = r.status_code == 200
    except Exception:
        vllm_ok = False
    return {"status": "ok" if vllm_ok else "degraded",
            "vllm": "up" if vllm_ok else "down",
            "model": "sql-llama-8b-awq"}

@app.get("/metrics")
async def metrics():
    return PlainTextResponse(generate_latest(), media_type="text/plain; version=0.0.4")

@app.get("/evaluate")
async def evaluate():

```

```
"""Run inline regression suite. Returns execution accuracy."""  
# See Section 5 for full implementation  
return await run_regression_suite()
```

Section 5 — Evaluation Endpoint

```
# /evaluate – runs the Day 12 regression harness inline
# Returns execution accuracy across difficulty tiers

async def run_regression_suite() -> dict:
    TEST_CASES = [
        # easy
        {"q": "Count all users", "gold": "SELECT COUNT(*) FROM users", "diff": "easy"},
        {"q": "List users from USA", "gold": "SELECT * FROM users WHERE country='USA'", "diff": "easy"},
        # medium
        {"q": "Total revenue from completed orders", "gold": "SELECT SUM(amount) FROM orders WHERE status='complete'", "diff": "medium"},
        {"q": "Email and total spend per user", "gold": "SELECT u.email, SUM(o.amount) FROM users u JOIN orders o ON u.user_id=o.user_id GROUP BY u.email", "diff": "medium"},
        # hard
        {"q": "Users who have never placed an order", "gold": "SELECT u.email FROM users u LEFT JOIN orders o ON u.user_id=o.user_id WHERE o.order_id IS NULL", "diff": "hard"},
    ]

    conn = sqlite3.connect(":memory:")
    conn.executescript(SCHEMAS["ecommerce"])
    # insert sample rows ...

    by_diff = {}
    for case in TEST_CASES:
        gen = await call_vllm(case["q"], SCHEMAS["ecommerce"])
        try:
            pred = set(tuple(r) for r in conn.execute(gen["sql"]).fetchall())
            gold = set(tuple(r) for r in conn.execute(case["gold"]).fetchall())
            ok = pred == gold
        except Exception:
            ok = False
        by_diff.setdefault(case["diff"], []).append(ok)

    conn.close()
    overall = [v for vals in by_diff.values() for v in vals]
    return {
        "accuracy": sum(overall) / len(overall),
        "by_difficulty": {k: sum(v)/len(v) for k,v in by_diff.items()},
        "n": len(overall),
    }

# Typical output:
# {"accuracy": 0.80, "by_difficulty": {"easy": 1.0, "medium": 0.75, "hard": 0.5}, "n": 5}
```

Section 6 — SLO Benchmark

Run after every model or config change. Confirms the system meets production SLOs before traffic is allowed.

SLO	Target	What It Measures
TTFT p50	< 200ms	Median first-token latency — smoothness of streaming

TTFT p95	< 500ms	Tail first-token latency — worst normal case
E2E p50	< 500ms	Median full SQL generation time
E2E p99	< 2000ms	Tail full SQL time — catches outlier long queries
Error rate	< 1%	HTTP 5xx from vLLM or Python exceptions
Syntax valid	> 97%	sqlparse reports valid SQL statement

```

import asyncio, httpx, time, statistics

SLOs = {"ttft_p95_ms":500, "e2e_p99_ms":2000,
        "error_rate_pct":1, "syntax_ok_pct":97}

async def benchmark(n_requests=100, concurrency=10):
    sem = asyncio.Semaphore(concurrency)
    results = []
    async with httpx.AsyncClient(base_url="http://localhost:9000", timeout=15) as client:
        async def one(q):
            async with sem:
                t0 = time.perf_counter()
                try:
                    r = await client.post("/sql", json={"question":q})
                    ms = (time.perf_counter()-t0)*1000
                    if r.status_code == 200:
                        d = r.json()
                        return {"ok":True, "e2e_ms":ms, "ttft_ms":d["ttft_ms"], "valid":d["valid_syntax"]}
                    except Exception: pass
                return {"ok":False, "e2e_ms":(time.perf_counter()-t0)*1000}
        results = await asyncio.gather(*[one(q) for q in QUESTIONS*10])

    ok = [r for r in results if r.get("ok")]
    errs = len(results) - len(ok)
    e2e = sorted(r["e2e_ms"] for r in ok)
    ttft = sorted(r["ttft_ms"] for r in ok if "ttft_ms" in r)
    p = lambda lst, q: lst[int(q*len(lst))] if lst else 0

    report = {"ttft_p95_ms":p(ttft, .95), "e2e_p99_ms":p(e2e, .99),
              "error_rate_pct":errs/len(results)*100,
              "syntax_ok_pct":sum(r.get("valid", 0) for r in ok)/len(ok)*100}

    for k, v in report.items():
        slo = SLOs[k]
        flag = "PASS" if v <= slo else "FAIL"
        print(f" {k:<22} {v:>7.1f} SLO={slo} [{flag}]")
    return report

asyncio.run(benchmark())

```


Section 7 — Prometheus + Grafana Monitoring

```
# docker-compose.yml
version: "3.8"
services:
  prometheus:
    image: prom/prometheus:latest
    ports: ["9090:9090"]
    volumes: ["/prometheus.yml:/etc/prometheus/prometheus.yml"]

  grafana:
    image: grafana/grafana:latest
    ports: ["3000:3000"]
    environment: [GF_SECURITY_ADMIN_PASSWORD=admin]
    depends_on: [prometheus]

# prometheus.yml
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: sql-api
    static_configs: [{targets: ["host.docker.internal:9000"]}]}
  - job_name: vllm
    static_configs: [{targets: ["host.docker.internal:8000"]}]}
metrics_path: /metrics
```

Key Grafana / PromQL Queries

Dashboard Panel	PromQL
P95 TTFT (last 5m)	histogram_quantile(0.95, rate(sql_ttft_seconds_bucket[5m]))
Request rate	rate(sql_requests_total[1m])
Error rate %	rate(sql_requests_total{status='error'}[5m]) / rate(sql_requests_total[5m]) * 100
Active requests	sql_active_requests
P99 E2E latency	histogram_quantile(0.99, rate(sql_latency_seconds_bucket[5m]))
Syntax error rate	rate(sql_syntax_errors_total[5m])
Median output tokens	histogram_quantile(0.50, rate(sql_output_tokens_bucket[5m]))

Section 8 — Production Readiness Checker

```
import asyncio, httpx, sys

async def check_all(url="http://localhost:9000"):
    checks = {}
    async with httpx.AsyncClient(base_url=url, timeout=10) as c:
        # 1. Health
        try:
            r = await c.get("/health"); d = r.json()
```

```

        checks["health_endpoint"] = r.status_code == 200
        checks["vllm_reachable"] = d.get("vllm") == "up"
    except: checks["health_endpoint"] = checks["vllm_reachable"] = False

# 2. SQL generation
try:
    r = await c.post("/sql", json={"question": "Count all users"})
    d = r.json()
    checks["sql_generation"] = r.status_code == 200
    checks["valid_sql_output"] = d.get("valid_syntax", False)
    checks["latency_under_2s"] = d.get("latency_ms", 9999) < 2000
except: checks["sql_generation"] = checks["valid_sql_output"] = False

# 3. Metrics endpoint
try:
    r = await c.get("/metrics")
    checks["metrics_endpoint"] = "sql_requests_total" in r.text
except: checks["metrics_endpoint"] = False

# 4. Regression accuracy
try:
    r = await c.get("/evaluate")
    acc = r.json().get("accuracy", 0)
    checks["regression_pass_90pct"] = acc >= 0.90
except: checks["regression_pass_90pct"] = False

all_pass = all(checks.values())
for name, ok in checks.items():
    print(f" [{ 'OK' if ok else 'FAIL' }] {name}")
print("\n" + ("READY" if all_pass else "NOT READY – fix failures"))
if not all_pass: sys.exit(1)

asyncio.run(check_all())

```

Section 9 — FAANG Production Deployment Patterns

Meta — Gradual Traffic Ramp with Automatic Rollback

Meta's production ML deployments follow a strict ramp: 1% → 5% → 10% → 25% → 50% → 100%. Each stage has a minimum dwell time (5 minutes internal tools, 30 minutes user-facing). A service called Opertune queries Manifold (Meta's metrics store) every 60 seconds: if error rate > 1% OR p95 latency > 2× baseline → automatic rollback to previous stage. The ramp advance and rollback decisions are fully automated — no human in the loop.

Google — Binary Canary with Statistical Comparison

Google's deployment unit is a binary (compiled model + serving code). A canary binary serves 1% of queries while stable serves 99%. Quality metrics are compared statistically against the stable binary (not fixed SLOs). If p95 latency delta > 10% OR quality delta > 0.5% → canary fails automatically. For Vertex AI LLM deployments: consistent user hashing ensures the same user always hits the same binary, preventing split-experience bugs during canary.

Amazon — Traffic Splitting with SageMaker Production Variants

Amazon SageMaker supports production variants: two model versions behind one endpoint URL with configurable traffic split (95% variant A, 5% variant B). CloudWatch alarms on ModelLatency, Invocation4XXErrors, Invocation5XXErrors trigger automatic traffic shift back to 100% A if thresholds breach. After 24 hours at 5% with no alarms → automated approval promotes B to 50%.

Netflix — Shadow Mode Before Any Live Traffic

Netflix's highest-risk deployments go through shadow mode: the new model receives a copy of every production request but responses are discarded — no user impact. Shadow mode collects latency distributions and LLM-judge quality scores. If shadow metrics pass the bar, the model enters canary. Critical for Netflix: a recommendation quality regression affects 260M users simultaneously — there is no small blast radius. Shadow mode eliminates unknown unknowns before exposure.

Microsoft — Blue-Green with Atomic Feature Flag Flip

Microsoft Copilot deployments use blue-green: a complete parallel environment (green) runs the new model while blue serves 100% of production traffic. Regression suites and LLM-judge evaluations run against green. When green passes all gates, a feature flag in Azure App Configuration flips traffic: green becomes production, blue becomes standby — atomically across all Azure regions. Rollback = flip the flag back. ZeRO-Inference enables 70B serving on single A100 via CPU/NVMe KV cache offload for cost-sensitive batch workloads.

Section 10 — Text-to-SQL Capstone: Pieces 1–5

Piece	Day	What	Status
1	Day 6	SQLOutput Pydantic schema + instructor retry loop	DONE
2	Week 3+	Schema-aware RAG (unknown tables)	UPCOMING
3	Week 5	Self-correcting agent (retry on syntax error)	UPCOMING
4	Day 12	Evaluation harness (execution accuracy by tier)	DONE
5	Day 14	Efficient serving stack (AWQ + vLLM + FastAPI)	DONE TODAY
6	Week 10	Security layer (SQL injection guard)	UPCOMING
7	Week 11	LLMOps dashboard (drift, accuracy over time)	UPCOMING
8	Week 12	Full end-to-end capstone presentation	UPCOMING

Current System Capabilities

SERVING STACK CAPABILITIES (end of Day 14):		
Natural language -> validated SQL		DONE (/sql endpoint)
AWQ INT4 quantized model: 4 GB vs 16 GB FP16		DONE
Prefix caching: 95% TTFT savings		DONE (vLLM flag)
Chunked prefill: -60% p95 TTFT		DONE (vLLM flag)
FP8 KV cache: 2x concurrent capacity		DONE (vLLM flag)
Prometheus metrics: latency, tokens, errors		DONE (/metrics)
Regression evaluation: execution accuracy gate		DONE (/evaluate)
SLO benchmark: TTFT/E2E/error gate		DONE (benchmark.py)
Production readiness gate: 6-check pass/fail		DONE (readiness.py)
Canary/shadow deploy plan: documented per FAANG		DONE (Section 9)
STILL MISSING (upcoming pieces):		
Schema-aware RAG (handle unknown tables)		Week 3
Self-correction loop (auto-retry on error)		Week 5
SQL injection guard (security layer)		Week 10
Multi-tenant schema registry		Week 10
LLMOps drift detection (accuracy over time)		Week 11

Section 11 — Architect's Lens: Production Readiness Checklist

PRODUCTION READINESS CHECKLIST (all must pass before live traffic):	
CORRECTNESS	
[] Regression suite (/evaluate) >= 90% execution accuracy	
[] Syntax error rate < 3% (SYNTAX_ERRORS metric)	
[] LLM-judge score >= 4/5 on 20-query sample	
PERFORMANCE (verified by benchmark.py)	

- [] TTFT p50 < 200ms (prefix cache hit rate > 80%)
- [] TTFT p95 < 500ms
- [] E2E p50 < 500ms for typical SQL queries (< 200 output tokens)
- [] E2E p99 < 2000ms
- [] TPOT p50 < 30ms (>= 33 tokens/sec streaming)

RELIABILITY

- [] /health returns 200 with vllm=up
- [] vLLM has restart=always (systemd or Docker policy)
- [] Error rate < 1% at target concurrency (tested in benchmark)

OBSERVABILITY

- [] Prometheus scraping /metrics every 15s
- [] Grafana panels: TTFT p95, error rate, active requests, accuracy
- [] Alert defined: TTFT p95 > 1000ms for > 2min -> page on-call
- [] Alert defined: error_rate > 2% for > 1min -> page on-call

DEPLOYMENT

- [] readiness.py passes all 6 checks
- [] Rollback plan documented (revert vLLM config < 5 minutes)
- [] Canary plan: start at 5%, hold 30 min, monitor SLOs, ramp

WEEK 2 INTEGRATION: Every day fed directly into this project. Day 8 (quantization) → AWQ weights. Day 9 (LoRA) → fine-tuned base model. Day 10 (full FT) → decision: LoRA sufficient, full FT cost not justified. Day 11 (serving) → vLLM continuous batching + PagedAttention. Day 12 (evaluation) → /evaluate endpoint with execution accuracy. Day 13 (efficiency) → prefix caching + chunked prefill + FP8 KV. Day 14 → FastAPI + Prometheus + SLO gate + deployment checklist. This is what production AI engineering looks like.

Section 12 — Hands-On Exercises

Exercise 1: Launch the Full Stack

```
# Terminal 1 — start vLLM
bash serve.sh

# Terminal 2 — start FastAPI
uvicorn api:app --port 9000 --workers 1

# Terminal 3 — smoke test
curl http://localhost:9000/health
curl -X POST http://localhost:9000/sql \
  -H "Content-Type: application/json" \
  -d '{"question":"List top 5 users by total spend","execute":true}'
curl http://localhost:9000/evaluate
curl http://localhost:9000/metrics | grep sql_requests
```

Exercise 2: Find Your Concurrency Cliff

```
# Run benchmark at increasing concurrency until SLOs break
for concurrency in [1, 5, 10, 20, 50, 100]:
    result = asyncio.run(benchmark(concurrency=concurrency))
    print(f"C={concurrency:3d}: TTFT p95={result['ttft_p95_ms']:.0f}ms "
          f"E2E p99={result['e2e_p99_ms']:.0f}ms "
          f"RPS={result['throughput_rps']:.1f}")
# Find where TTFT p95 > 500ms -> that is your auto-scaling trigger
```

Exercise 3: Add Request Tracing

```
import uuid
from fastapi import Request

@app.middleware("http")
async def add_request_id(request: Request, call_next):
    req_id = request.headers.get("X-Request-ID", str(uuid.uuid4()))
    response = await call_next(request)
    response.headers["X-Request-ID"] = req_id
    return response

# Every response now carries X-Request-ID
# Clients log it; you correlate frontend errors to backend logs
# grep -r "X-Request-ID: <uuid>" /var/log/sql-api/* -> full trace
```

Summary — Day 14 and Week 2 Complete

What You Built	How
AWQ INT4 model (4 GB)	autoawq.quantize() with 160-sample calib data
vLLM production server	All Day 13 flags: prefix cache, chunked prefill, FP8 KV
FastAPI /sql endpoint	Schema lookup + vLLM call + syntax check + metrics
FastAPI /health endpoint	Checks vLLM reachability, returns status JSON
FastAPI /metrics endpoint	Prometheus Counter/Histogram/Gauge via generate_latest()
FastAPI /evaluate endpoint	Inline regression suite, execution accuracy by difficulty
SLO benchmark	Async concurrent load test, TTFT/E2E/error/syntax
Prometheus + Grafana	docker-compose stack, PromQL panels, latency alerts
Production readiness checker	6-check automated gate before any deployment
FAANG deploy patterns	Meta ramp, Google canary, Amazon variants, Netflix shadow

NEXT: Week 3 — RAG Engineering (Day 15)

Days 15-21 cover the retrieval layer: production indexing pipelines, chunking strategies at scale, embedding model selection and fine-tuning, hybrid search in production, re-ranking, RAG evaluation (RAGAS), and Week 3's project — building the schema RAG layer for the SQL model (Capstone Piece 2: handle unknown tables via retrieval).

Eagle Eye Addendum — Day 14: SPIDER Benchmark + NL2SQL

SPIDER is the primary benchmark for Text-to-SQL (NL2SQL) evaluation. It contains 10,181 questions across 200 databases with 138 domains. NL2SQL (Natural Language to SQL) is the formal task name for converting a natural-language question into an executable SQL query.

```
# SPIDER benchmark – standard for NL2SQL evaluation
# Source: Yale NLP, 2018. 10,181 NL question → SQL pairs.
# 200 databases, 138 domains, avg 5.1 tables per database.

# Difficulty split:
# Easy: 27% – single table, no join, simple conditions
# Medium: 39% – 1-2 joins, GROUP BY or HAVING
# Hard: 20% – 3+ joins, subqueries, set operations
# Extra: 14% – nested subqueries, INTERSECT/EXCEPT

# Key metric: Execution Accuracy (EX)
# EX = fraction of generated SQLs that return the same result set
#       as the gold SQL when executed on the database.
# (Different from exact string match – two different SQL strings
#  can produce the same result and both be "correct")

# NL2SQL pipeline (this capstone):
# NL question → schema retrieval (RAG) → SQL generation (LLM) → execute

# State-of-the-art EX on SPIDER (2024):
# GPT-4 + few-shot: 86.6%
# Fine-tuned CodeLlama-34B: 83.1%
# Our production target: > 82% EX (Day 14 SLO)
```